SSR: Symbolic-Neural Soft Reasoning for Robust and Verifiable Thinking Chains

Rui Wang[1] **Zeming Wei**[1*] **Yihao Zhang**[1] **Xiaokun Luan**[2]

[1]Peking University [2]National University of Singapore

{2300010607,weizeming,zhangyihao}@stu.pku.edu.cn xiaokun.luan@nus.edu.sg

Abstract

Large Language Models (LLMs) have demonstrated impressive progress in complex reasoning tasks, largely driven by the Chain-of-Thought (CoT) paradigm, which decomposes difficult problems into intermediate steps. However, CoT reasoning remains fundamentally constrained by the probabilistic nature of neural generation, leading to unfaithful reasoning chains that undermine reliability. Neuro-symbolic approaches attempt to address these issues by combining LLMs with symbolic solvers, yet they face persistent challenges, including hallucinated translations, the mismatch between natural language and formal logic, and the limited enhancement of the LLM's intrinsic reasoning ability. To overcome these limitations, we propose Symbolic-Neural Soft Reasoning (SSR), a unified framework that integrates LLMs with symbolic reasoning and improves robustness by relaxing strict logical determinism while preserving verifiability. Our approach improves reasoning performance, automatically generates verifiable and human-readable logical thinking chains for training and fine-tuning, and facilitates cross-disciplinary applications such as AI for mathematics. Experiments across multiple models and benchmarks demonstrate that SSR consistently outperforms existing reasoning frameworks, highlighting its effectiveness in enhancing both the robustness and interpretability of LLM reasoning. Our code is available at https://github.com/1240148048/SSR.

1 Introduction

Large Language Models (LLMs) have achieved remarkable progress on complex reasoning tasks, exemplified by recent systems such as DeepSeek-R1 (Guo et al., 2025) and GPT-4o (Hurst et al., 2024). Reasoning in LLMs broadly refers to the ability to maintain logically consistent internal states, integrate world knowledge, and perform multi-step deductions over long contexts (Plaat et al., 2024). However, vanilla autoregressive generation suffers from a fundamental *one-step bottleneck*: for complex multi-step reasoning problems, LLMs struggle to maintain a correct line of reasoning across a long sequence of dependent token predictions, often resulting in premature abandonment, random guessing, or getting trapped in local loops. (Arbuzov et al., 2025).

Chain-of-Thought (CoT) prompting (Wei et al., 2022) has emerged as a powerful paradigm to mitigate this issue by decomposing complex reasoning into explicit intermediate steps, effectively expanding the model's computational workspace and substantially improving performance, even with minimal prompting (Kojima et al., 2022). Building upon CoT, structured reasoning frameworks such as Tree of Thoughts (Yao et al., 2023) and Program of Thoughts (Chen et al., 2022) further enhance reasoning via branching exploration or external execution. Beyond accuracy gains, explicit reasoning chains also provide interpretability and valuable supervision signals for training and fine-tuning.

Despite these advantages, CoT-based reasoning remains fragile. Recent work has shown that reasoning chains may be unfaithful, serving as post-hoc rationalizations rather than reflecting the true causal process (Turpin et al., 2023), and are susceptible to confirmation bias (Wan et al., 2025), where models tend to reinforce their initial priors during the reasoning process. More fundamentally, LLM reasoning inherits hallucination issues rooted in probabilistic generation, including fabrication, incorrect inference, and misuse of rules (Sun et al., 2025b). These issues compromise both the reliability and educational value of LLM-based reasoning systems, even when final answers are correct.

To address these limitations, neuro-symbolic frameworks combine LLMs with symbolic solvers, leveraging the flexible natural language process-

*Corresponding author.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 28289–28308
October 24-29, 2026 ©2026 Association for Computational Linguistics

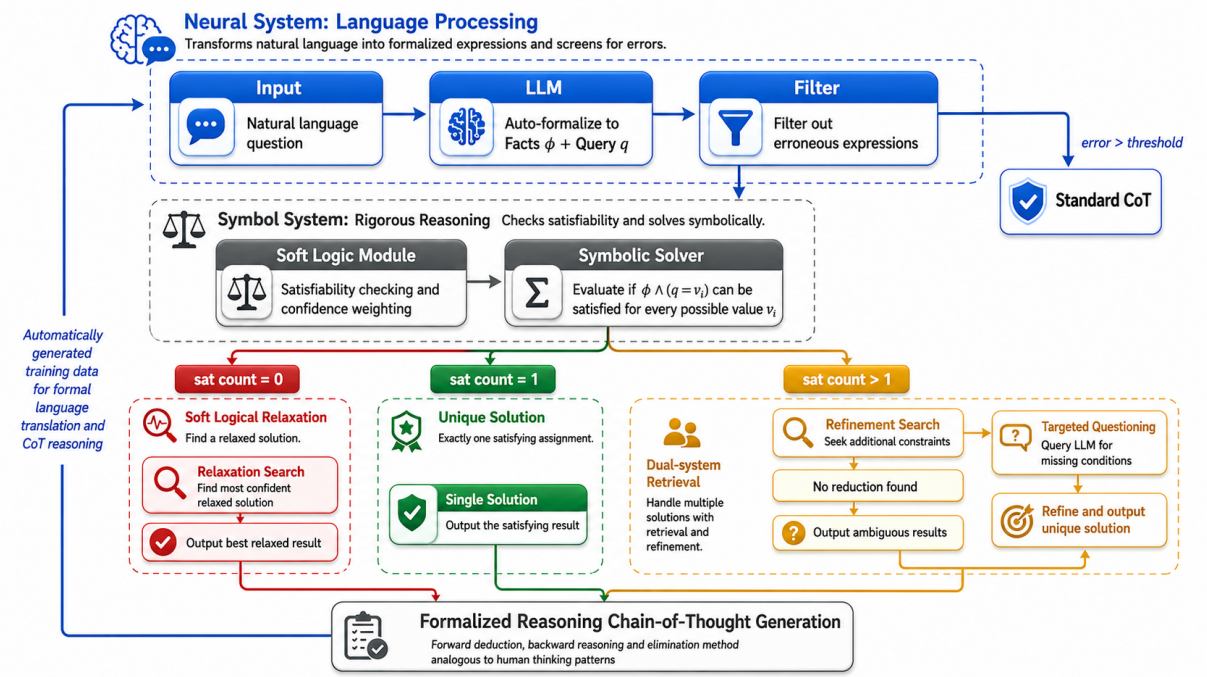


Figure 1: Overall Process and Application Framework Diagram of the SSR Method

ing capabilities of neural systems and the rigorous reasoning capabilities of symbolic systems (Hitzler et al., 2022; Hitzler and Sarker, 2022). In this paradigm, LLMs translate natural language into symbolic constraints, which are then solved deterministically. While effective in improving correctness, this approach introduces new challenges. Auto-formalization itself is prone to hallucinations, omissions, and structural errors (Weng et al., 2025). Moreover, the inherent ambiguity and context dependence of natural language conflict with the strict semantics required by formal logic, often leading to brittle or unsatisfiable formulations. Finally, delegating the core reasoning process entirely to symbolic solvers raises concerns about whether such systems genuinely enhance the LLM's intrinsic reasoning ability, rather than merely bypassing it (Pan et al., 2023; Feng et al., 2025a).

In this work, we argue that these limitations stem from treating symbolic reasoning as strictly deterministic, leaving no tolerance for uncertainty or imperfect formalization. We propose **Symbolic-Neural Soft Reasoning (SSR)**, a neuro-symbolic framework that integrates *soft satisfiability* constraints to enhance robustness under noisy or uncertain premises. The key premise of SSR is that an LLM-generated formalization should be treated as noisy evidence rather than as a perfect logical program. SSR uses solver feedback to select among three regimes: hard SMT inference when the translation is consistent and decisive, confidence-weighted MaxSAT-style relaxation when it is inconsistent, and solver-guided targeted retrieval when it is underdetermined. Our framework aims to (i) improve reasoning accuracy of existing LLMs, (ii) identify and conduct targeted retrieval of reasoning chain breakpoints via neuro-symbolic collaboration, and automatically generate faithful and interpretable logical reasoning chains for LLM training and coordinated development of neuro-symbolic systems, and (iii) support practical applications such as planning and finite discrete mathematical reasoning. Experimental results across multiple models and benchmarks demonstrate the effectiveness of SSR on both reasoning accuracy and reasoning-chain quality. Figure 1 illustrates the overall SSR pipeline.

2 Methodology

SSR aims to combine the semantic flexibility of LLMs with the consistency guarantees of symbolic solvers. Given a natural language problem, an LLM first abstracts it into a restricted logical program, and an SMT solver then performs verifiable inference over the translated constraints. Since LLMs may introduce errors when translating formal languages, or the inherent properties of the questions prevent a unique answer from being obtained, SSR further introduces soft logical relaxation and iden-

tifies chain-breaking points in dual-system interactive retrieval reasoning. Moreover, SSR can generate formalized reasoning chains to be translated into concise natural-language reasoning chains or used as training data.

2.1 Hard Logical Reasoning

We first consider an idealized setting where the LLM translates the natural language input into formal logic without ambiguity or error. This setting isolates the core inference problem: once the symbolic program is correct, reasoning can be reduced to deterministic SMT solving. Detailed abstraction rules are provided in Appendix A.

Given a problem, the LLM produces a set of symbolic expressions consisting of facts and queries. Facts encode known constraints, while queries specify the target predicate functions to be inferred. Predicate functions represent object properties and relations, and are categorized into Numeric Sort or Boolean Sort according to whether their domains are finite enumerable values or truth values, such as $\mathrm{Age}(\mathrm{Bob}) = 16$ and $\mathrm{Happy}(\mathrm{Anne})$. All expressions follow a restricted SMT-compatible syntax to ensure consistency and verifiability.

Let the fact set be:

$$\Phi = \bigwedge_{i=1}^{n} \varphi_i. \tag{1}$$

Let $Q = \{q_1, \ldots, q_m\}$ be the query set. Reasoning is performed by checking satisfiability under different query conditions, as detailed in Appendix B. For a Boolean query q_i, SSR checks $\Phi \wedge q_i$ and $\Phi \wedge \neg q_i$. If only one case is satisfiable, the corresponding truth value is returned; if both are satisfiable, the result is Unknown.

For a numeric query with free predicate variable a_i, where $a_i \in \{a_{i1}, a_{i2}, \ldots, a_{it_i}\}$, SSR checks each candidate assignment $\Phi \wedge (a_i = a_{ij})$. The solution set is:

$$\{v \in \mathcal{D}_i \mid \Phi \wedge (a_i = v) \text{ is } \textbf{sat}\}. \tag{2}$$

Arithmetic reasoning is handled by translating arithmetic operators into symbolic expressions.

Under the idealized assumption, Φ is satisfiable and at least one query value is satisfiable. Thus, hard logical reasoning provides a rigorous deterministic foundation: inference is delegated to the solver, which enforces global consistency and avoids hallucinated reasoning. Its limitation is equally clear: if the LLM-generated logic is noisy, incomplete, or over-constrained, hard logic becomes brittle.

2.2 MaxSAT Relaxation

Hard logic fails when translated constraints are not perfectly reliable. In LLM-assisted reasoning, errors may arise from omitted premises, imprecise predicates, missing negations, or implicit commonsense assumptions. Some real-world conditions are also approximate or context-dependent. SSR therefore relaxes hard constraints into confidence-weighted MaxSAT, allowing symbolic reasoning to proceed under noise and uncertainty.

MaxSAT is an optimization variant of the Boolean Satisfiability (SAT) problem that seeks an assignment maximizing the number of satisfied clauses, supporting hard and soft constraints to enable logical relaxation for handling unsatisfiable or unreliable logical conditions (Dubois and Prade, 1998; Kimmig et al., 2012). SSR adopts this relaxation to tolerate translation errors, missing premises, and uncertain real-world knowledge (Kimmig et al., 2012).

Before solving, SSR applies a syntax-level filter to the LLM-generated expressions. Clearly malformed expressions, such as raw natural-language fragments, invalid predicate usage, or garbled characters, are replaced by `BoolVal(True)`, a non-informative placeholder that prevents parser failure. If Φ contains more than one such placeholder, or if the query q_i is replaced by `BoolVal(True)`, the translation is treated as severely corrupted and SSR falls back to a conventional CoT pipeline.

For the remaining facts, each $\varphi \in \Phi$ receives a confidence weight $w(\varphi) > 0$. We use either a uniform prior or an entropy-based certainty score:

$$w(\varphi) = \exp(-H(\varphi)), \tag{3}$$

where $H(\varphi)$ is the length-normalized entropy of the LLM output associated with φ, computed from token-level logit probabilities. This is because it is negatively correlated with internal uncertainty and measures how much the model trusts a constraint when the fact set is in tension. Entropy-based weights are used only when token-level output probabilities are available from the model interface; otherwise, SSR uses uniform weights. Table 5 reports their comparison on Qwen2.5-7B-Instruct.

Given weighted facts $\{\varphi_1, \ldots, \varphi_n\}$ and query q_i, SSR enumerates candidate values q_{ij}. For closed logical queries, the candidates are $\{\mathrm{True}, \mathrm{False}\}$;

for variable queries, they are values in the domain. SSR first checks whether Φ is satisfiable. If Φ is `unsat`, it searches for a satisfiable subset with the largest total confidence weight.

After relaxation, if exactly one candidate is `sat`, SSR returns it as in hard logic. If no candidate is `sat`, then for each candidate $(q_i = q_{ij})$, SSR retrieves a subset $\Phi'_j \subseteq \Phi$ such that $\Phi'_j \wedge (q_i = q_{ij})$ is `sat` and the total confidence weight of Φ'_j is maximized. SSR selects the candidate associated with the maximum-weight satisfiable subset.

Thus, soft relaxation keeps SMT satisfiability as the validity criterion while replacing all-or-nothing failure with confidence-weighted constraint selection in real-world scenarios where formal logic may not be perfect.

2.3 Dual-System Targeted Retrieval under Ambiguity

Soft relaxation resolves inconsistency, but it may still leave multiple candidates satisfiable. This may stem from translation omissions and errors, or the absence of decisive conditions caused by unrecognized implicit information in questions or commonsense knowledge. SSR handles this ambiguity through targeted retrieval: the solver first identifies which predicate assignment would make the answer unique, and the LLM then verifies whether that assignment is supported by the problem statement or commonsense knowledge.

When more than one candidate is `sat`, SSR enumerates possible predicate-function values and searches for an assignment $a = \text{value}$ such that fixing it makes exactly one candidate satisfiable:

$$\Phi \wedge (a = \text{value}) \wedge (q_i = q_{ij}). \tag{4}$$

If no such assignment exists, SSR retains the original results. If such an assignment exists, SSR further searches for another assignment $a' = \text{value}'$ that can derive it. Specifically, SSR checks whether $\Phi \wedge (a' = \text{value}') \wedge (a = \text{value})$ is `sat` while $\Phi \wedge (a' = \text{value}') \wedge \neg(a = \text{value})$ is `unsat`. If so, $a = \text{value}$ is replaced by $a' = \text{value}'$, and the search continues iteratively until no further retrievable assignment exists. Essentially, this recursion conducts backward search for required information to identify conditions that can connect coherently with forward reasoning, thereby completing the reasoning chain.

The discriminator search is finite. Values are enumerated from the finite candidate domain of the instance. The current implementation permits at most one backward replacement by a premise that can support the discriminator. If no discriminating assignment is found, or the LLM cannot confirm the localized premise from the input, SSR preserves the original candidate set and returns UNKNOWN hen a satisfiable assignment contains both True and False values. Meanwhile, such targeted retrieval is only capable of handling inference chains with a single broken logical condition. For this reason, we filter instances where formal translation yields more than one apparent error and delegate these cases to Chain-of-Thought.

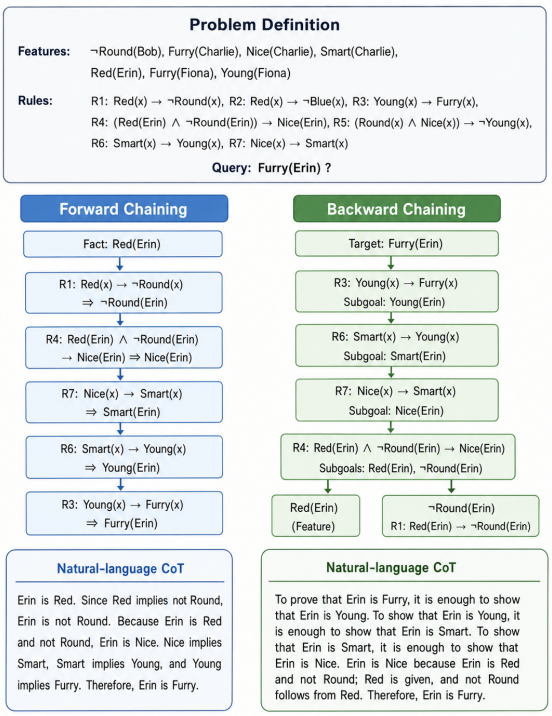


Figure 2: Example reasoning chains generated via forward and backward deduction in the SSR framework.

2.4 Human-Readable Chain-of-Thought Auto-Generation

SMT solving yields verifiable answers but not human-readable reasoning traces. To improve interpretability and practicality, SSR constructs symbolic reasoning chains from the verified logical program. The generated explanation is therefore grounded in symbolic inference rather than randomly generated by the LLM.

SSR first normalizes symbolic inputs into properties and rules. A property specifies that a predicate assignment is given or excluded, such as Predicate(Object) or ¬Predicate(Object). A rule encodes an implication between properties, such as

$P \rightarrow Q$. Quantified statements are expanded into conjunctions or disjunctions of simple rules. If the query itself is a rule, its premise P is temporarily added to the fact set and Q becomes the new target.

Backward reasoning starts from the query target and retrieves rules whose conclusions match it. The premises of these rules become sub-goals, and the process repeats until the target is supported by given properties or no further rule applies. If the query is determined by excluding alternatives, this becomes backward reasoning with elimination, which subsumes "proof by contradiction".

Forward reasoning starts from known properties and iteratively applies rules whose premises are satisfied. Newly derived properties are added to the knowledge set and may trigger further rules. The process stops when the query predicate is determined, either directly or by elimination, or when no new property can be derived.

The resulting forward and backward traces form explicit symbolic chains. These chains can be stored as upright and inverted n-ary trees, respectively, translated by an LLM into natural-language CoT, grouped by reasoning pattern, and can be automatically verified, with the shortest paths selected separately as the standard answers for different methods. Besides interpretability, this provides scalable and verifiable reasoning traces, reducing dependence on costly human annotation (Ahn et al., 2024) and addressing a limitation of prior neuro-symbolic systems, where symbolic reasoning improved accuracy but did not help the model improve its intrinsic reasoning capability (Pan et al., 2023).

Overall, SSR follows a progressive design: neural system processes diverse languages, symbolic system provides deterministic inference, soft reasoning addresses the fragility of formal deduction, targeted retrieval resolves ambiguity and clue breakage, and symbolic CoT generation exposes the reasoning process in a verifiable human-readable form.

3 Experiments

3.1 Experimental Setup

Models. We evaluate our method using a diverse set of LLMs. Specifically, we include two strong open-source instruction-tuned models, **Qwen2.5-7B-Instruct** (Yang et al., 2025) and **DeepSeek-R1** (Guo et al., 2025), as well as the proprietary model **GPT-4o** (Hurst et al., 2024) and **GPT-5.4** (Singh et al., 2025). This selection allows us to assess the effectiveness of our framework across models with different architectures and scales.

Datasets. We conduct experiments on multiple benchmark datasets covering a broad spectrum of logical reasoning tasks. **PrOntoQA** (Saparov and He, 2022) is used to evaluate deductive reasoning capabilities under well-structured logical settings. While **ProofWriter** (Tafjord et al., 2021) extends deductive reasoning by introducing an *unknown* option, testing whether LLMs can correctly assess the sufficiency of given conditions. **FOLIO** (Han et al., 2024) contains more complex first-order logic reasoning problems, where long and syntactically complex sentences as well as implicit commonsense assumptions pose significant challenges to LLM-based reasoning. Finally, **LogicalDeduction** is a logical reasoning task from the BigBench benchmark (Ghazal et al., 2013) used to evaluate constraint satisfaction problems in discrete combinatorial scenarios.

Baselines. We compare our proposed **SSR** method against several representative baselines. These include **Direct** answering, where the model generates an answer without explicit reasoning; standard Chain-of-Thought (**CoT**) (Wei et al., 2022) prompting; **Logic-LM** (Pan et al., 2023), which translates natural language into formal logic and delegates reasoning to a symbolic solver; and **Vери-CoT** (Feng et al., 2025a), which first generates a natural-language CoT and then verifies it step by step using a symbolic solver. These baselines cover classical prompt-engineering approaches and two representative neuro-symbolic reasoning frameworks. In addition, we introduce a self-designed **Formal CoT** baseline, where the LLM is prompted to generate a purely formal logical reasoning chain. We do not include other popular paradigms such as implicit reasoning or supervised fine-tuning, as these approaches are not readily applicable to closed-source models.

Evaluation Protocol. We set the maximum token generation length to 1000 and use the *Z3-solver* (De Moura and Bjørner, 2008) as the symbolic reasoning backend. For each dataset, we sample 200 problems for evaluation. To measure improvements in LLM reasoning performance, we compare the final-answer accuracy and the total inference time across all methods. For evaluating generated reasoning chains, a chain is counted as fully correct only if (i) its final answer matches the gold answer, (ii) every step can be replayed from an input fact, an explicitly marked retrieved

premise, or an earlier derived statement, and (iii) the chain terminates without an unsupported inference or cycle. Since the Direct and Logic-LM baselines do not produce explicit reasoning chains, we only compare reasoning-chain quality against **Formal CoT** and **VeriCoT**.

3.2 Main Results

Overall Superiority of SSR in Enhancing Reasoning Accuracy. As shown in Table 1, SSR improves over standard CoT on every model–dataset pair. Notably, SSR achieves strong gains even when compared to advanced neuro-symbolic baselines such as Logic-LM and VeriCoT, indicating that its advantages cannot be attributed solely to verification or symbolic execution. These results suggest that the soft relaxation integration in SSR enables more robust and flexible multi-step reasoning, effectively mitigating the stochastic and error-accumulation issues inherent in auto-formalization. Overall, the consistent dominance of SSR across models and reasoning tasks highlights its ability to enhance both reasoning accuracy and cross-dataset generalization. For detailed case analyses and discussions of the underlying causes of failures, please refer to Appendix E.

Analysis of the Baselines. As shown in the table, CoT reasoning significantly improves accuracy compared to directly producing final answers. In contrast, Formal CoT often underperforms standard CoT, suggesting that LLMs are more adept at reasoning in natural language than in symbolic logic since the inherently probabilistic nature of LLM generation remains unchanged. Logic-LM achieves notable overall improvements over CoT, while VeriCoT generally benefits from verification feedback to enhance final reasoning accuracy; however, VeriCoT cannot prevent cases where responses with incorrect reasoning but correct answers are modified into outputs with both incorrect reasoning and incorrect final answers.

Generating Correct and Verifiable Thinking Chains. Table 2 demonstrates that SSR is markedly more effective than existing baselines at generating fully correct and verifiable reasoning thought chains across different models and datasets. While VeriCoT rely on post-hoc verification of generated outputs, SSR directly integrates retrieval and verification into the reasoning loop, substantially increasing the likelihood that the generated thought chain is consistent and logically complete. Moreover, it is worth noting that the forward deduction approach is relatively more stable, which may be because retrieving relevant premises based on the goal during backward deduction is more prone to errors. Overall, these results confirm that deep neural–symbolic interaction in SSR enables LLMs to construct verifiable, end-to-end correct reasoning chains rather than merely producing plausible but invalid explanations. This can automatically provide multi-path, verifiable chains of thought for LLM inference training.

3.3 Ablation Study and Comparative Study

Attribution of execution-branch accuracy. As shown in Table 3, the distributions of correct and incorrect answers between the symbolic execution branch and the CoT fallback branch in SSR exhibit statistically significant differences. In particular, samples successfully handled by symbolic execution tend to be far more reliable than those diverted to CoT due to logical translation failures, highlighting the effectiveness of formal verification when symbolic parsing succeeds. These results demonstrate that SSR's gains primarily stem from its ability to maximize successful symbolic execution, while the CoT branch serves as a necessary but less reliable backup. Moreover, within the SSR framework, samples processed via the CoT branch exhibit a lower accuracy than the overall accuracy of the standalone CoT method. This suggests that if an LLM finds it difficult to translate a problem into logical form, the problem is also statistically more challenging for the LLM to solve. In addition, we observe that the Z3 execution rate on the FOLIO dataset is relatively low, which partially explains why the performance advantage of SSR over CoT is less pronounced on FOLIO.

Robustness under Input Perturbations. In addition, we evaluate the robustness of SSR under controlled perturbations. Specifically, the setting 0 denotes the original input, while $+1$ and $+2$ indicate the injection of one or two distractor facts, respectively. Conversely, -1 and -2 correspond to the removal or modification of one or two key premises. For each selected premise, we randomly choose with equal probability whether to delete it or modify it; in the latter case, its value is replaced by another candidate function value sampled uniformly at random, e.g., changing $\mathrm{Pos}[\text{“Cat”}] = 2$ to $\mathrm{Pos}[\text{“Cat”}] = 3$. Figure 3 shows that, under the removal/modification settings, all methods experience performance degradation. Nevertheless, SSR consistently exhibits higher accuracy and a

Table 1: Overall reasoning accuracy (%) of SSR and baselines across different models and datasets. FCoT denotes Formal CoT, L-LM denotes Logic-LM, and V-CoT denotes VeriCoT. Results are reported as mean ± standard deviation over five runs.

Model	Dataset	Direct	CoT	FCoT	L-LM	V-CoT	SSR
Qwen-2.5-7B	PrOntoQA	$67.5_{\pm 0.00}$	$86.1_{\pm 0.37}$	$84.9_{\pm 0.20}$	$91.7_{\pm 0.24}$	$88.1_{\pm 0.20}$	$\mathbf{97.4}_{\pm 0.20}$
	ProofWriter	$37.0_{\pm 0.00}$	$58.1_{\pm 0.37}$	$50.9_{\pm 0.20}$	$77.9_{\pm 0.20}$	$73.4_{\pm 0.49}$	$\mathbf{86.0}_{\pm 0.32}$
	FOLIO	$55.5_{\pm 0.00}$	$64.5_{\pm 0.45}$	$56.1_{\pm 0.20}$	$62.2_{\pm 0.24}$	$\mathbf{66.0}_{\pm 0.32}$	$64.6_{\pm 0.20}$
	Logical Deduction	$46.0_{\pm 0.00}$	$63.2_{\pm 0.40}$	$41.8_{\pm 0.24}$	$72.0_{\pm 0.32}$	$66.5_{\pm 0.40}$	$\mathbf{75.0}_{\pm 0.32}$
DeepSeek-R1	PrOntoQA	$86.5_{\pm 0.00}$	$96.0_{\pm 0.00}$	$91.7_{\pm 0.24}$	$92.0_{\pm 0.32}$	$95.4_{\pm 0.20}$	$\mathbf{100.0}_{\pm 0.00}$
	ProofWriter	$56.0_{\pm 0.00}$	$72.5_{\pm 0.32}$	$73.5_{\pm 0.77}$	$89.0_{\pm 0.00}$	$80.1_{\pm 0.20}$	$\mathbf{98.9}_{\pm 0.20}$
	FOLIO	$58.5_{\pm 0.00}$	$68.0_{\pm 0.37}$	$56.2_{\pm 0.24}$	$69.5_{\pm 0.00}$	$64.4_{\pm 0.37}$	$\mathbf{71.5}_{\pm 0.32}$
	Logical Deduction	$74.5_{\pm 0.00}$	$84.0_{\pm 0.20}$	$83.5_{\pm 0.49}$	$88.7_{\pm 0.40}$	$85.6_{\pm 0.20}$	$\mathbf{90.2}_{\pm 0.24}$
GPT-4o	PrOntoQA	$83.5_{\pm 0.00}$	$94.5_{\pm 0.00}$	$86.0_{\pm 0.00}$	$88.5_{\pm 0.00}$	$96.5_{\pm 0.00}$	$\mathbf{100.0}_{\pm 0.00}$
	ProofWriter	$52.5_{\pm 0.00}$	$74.0_{\pm 0.37}$	$68.2_{\pm 0.24}$	$90.0_{\pm 0.00}$	$80.6_{\pm 0.20}$	$\mathbf{98.0}_{\pm 0.00}$
	FOLIO	$64.5_{\pm 0.00}$	$70.5_{\pm 0.32}$	$57.7_{\pm 0.24}$	$\mathbf{76.1}_{\pm 0.20}$	$66.2_{\pm 0.24}$	$75.2_{\pm 0.24}$
	Logical Deduction	$63.5_{\pm 0.00}$	$72.0_{\pm 0.00}$	$64.4_{\pm 0.49}$	$86.0_{\pm 0.00}$	$72.6_{\pm 0.24}$	$\mathbf{88.4}_{\pm 0.24}$
GPT-5.4	PrOntoQA	$97.0_{\pm 0.00}$	$98.5_{\pm 0.00}$	$90.0_{\pm 0.32}$	$\mathbf{100.0}_{\pm 0.00}$	$97.4_{\pm 0.24}$	$\mathbf{100.0}_{\pm 0.00}$
	ProofWriter	$79.5_{\pm 0.00}$	$87.1_{\pm 0.20}$	$78.4_{\pm 0.20}$	$95.0_{\pm 0.40}$	$89.2_{\pm 0.24}$	$\mathbf{98.6}_{\pm 0.58}$
	FOLIO	$69.5_{\pm 0.00}$	$74.5_{\pm 0.32}$	$64.8_{\pm 0.40}$	$76.4_{\pm 0.49}$	$72.3_{\pm 0.24}$	$\mathbf{78.1}_{\pm 0.49}$
	Logical Deduction	$71.0_{\pm 0.00}$	$76.6_{\pm 0.20}$	$69.8_{\pm 0.24}$	$88.7_{\pm 0.40}$	$79.9_{\pm 0.37}$	$\mathbf{90.0}_{\pm 0.32}$
Total Runtime (min)		35.7	265.6	288.6	362.0	451.5	**435.1**

Table 2: Overall experiment on the success rate of generating fully correct reasoning chains. We adopt abbreviations for the names of models and datasets, and "SSR-F", "SSR-B" refer to the SSR method using forward / backward deduction algorithms, respectively. All data represent the average accuracy over five runs.

Model	Dataset	FCoT	V-CoT	SSR-F	SSR-B
Qwen	PQ	21.9	51.7	**89.7**	88.5
	PW	14.5	32.3	**63.5**	62.0
	FO	10.8	17.9	**34.6**	31.1
	LD	22.6	34.6	**55.4**	50.8
DS	PQ	33.8	67.4	**94.5**	**94.5**
	PW	18.1	38.3	**73.1**	72.4
	FO	13.5	22.0	**39.4**	35.5
	LD	31.7	44.3	**69.7**	63.9
GPT-4o	PQ	34.6	66.7	**94.0**	93.9
	PW	20.7	39.7	**71.9**	70.2
	FO	15.4	23.0	**41.5**	36.1
	LD	30.1	42.7	**66.0**	61.2
GPT-5.4	PQ	36.6	68.3	**94.9**	94.2
	PW	22.9	39.5	**71.4**	70.4
	FO	15.2	23.4	**42.2**	38.0
	LD	32.5	44.3	**67.0**	63.6
Total Time (min)		389.6	630.4	708.5	595.3

smoother degradation trend than baseline methods, indicating stronger robustness to both distractor information and incomplete or corrupted premises. We also observe that SSR performs better than the baselines under the -1 setting, but this advantage becomes less pronounced under the -2 setting.

Ablation Study on Key Components.

Table 3: In the Qwen2.5-7B-Instruct model, we counted the number of right/wrong (r/w) samples processed by the Z3 symbolic solver, as well as the number of right/wrong samples processed by CoT due to excessive logical language translation errors in SSR. These data were then treated as a 2×2 contingency table to calculate the χ^2 values respectively. Note that one FOLIO example could not be parsed and is thus omitted.

Dataset	Symbol (r/w)	CoT (r/w)	χ^2
PrOntoQA	194/4	1/1	18.70
Proofwriter	149/9	23/19	43.09
FOLIO	74/19	63/43	9.36
LogicalDeduction	124/28	26/22	14.62

Table 4 shows that SSR's improvement cannot be attributed solely to selective routing to CoT. And retrieval is not always beneficial: removing it slightly improves ProofWriter accuracy from 98.6 to 99.0. We therefore regard targeted retrieval as an ambiguity-resolution mechanism, rather than a guaranteed correction step. Table 5 shows that removing the filtering module leads to consistent performance degradation across all datasets, indicating that explicitly eliminating obviously erroneous logical expressions is useful. In contrast, incorporating entropy-based confidence weighting in the soft reasoning module yields relatively modest improvements, suggesting that while uniform weighting already provides a strong baseline, adaptive weighting may mainly take effect in edge cases

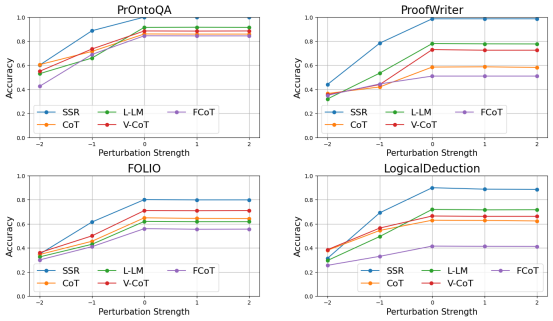


Figure 3: Comparative study on different datasets using the Qwen-2.5-7B-Instruct model. Where the horizontal axis (perturbation strength) denotes the number of randomly added perturbing words or phrase, and the vertical axis denotes reasoning accuracy.

Table 4: Final-answer accuracy (%) of component ablations with GPT-5.4. "-" denoted removing a component (MaxSAT relaxation, targeted retrieval, and CoT fallback) while leaving the remaining settings unchanged.

Dataset	SSR	- MaxSAT	- Retrieval	- Fallback
PQ	100.0	100.0	100.0	99.5
PW	98.6	98.5	99.0	97.0
FO	78.1	76.5	74.5	63.0
LD	90.0	89.5	89.5	84.5

where the uniform confidence weights corresponding to two candidate outcomes are the same. Overall, these results demonstrate that the mechanism designs in SSR play complementary roles.

4 Related Work

Logical Reasoning of LLMs and Soft Reasoning. LLMs have demonstrated strong empirical performance across a wide range of tasks, yet logical reasoning remains particularly challenging. Logical reasoning requires maintaining globally consistent symbolic states, correctly handling logical relations, dependencies, and sufficiency (Plaat et al., 2024). To address these challenges, prior work has explored several directions, including prompt-based methods such as chain-of-thought (CoT) (Wei et al., 2022), self-consistency (Wang et al., 2022), neuro-symbolic systems (Pan et al., 2023; Feng et al., 2025a), supervised fine-tuning (Luong et al., 2024; Yu et al., 2025), and implicit reasoning approaches that internalize reasoning processes without explicit intermediate steps (Hao et al., 2024; Shen et al., 2025).

Despite these advances, LLM-based logical reasoning remains fragile in the presence of noisy inputs, or intermediate reasoning errors, raising significant robustness concerns (Zhou et al., 2024). Existing robustness-oriented methods either focus on error localization and self-feedback, where models iteratively revise incorrect premises or reasoning steps based on verification signals (Sun et al., 2025a), or adopt soft logical relaxation (Dubois and Prade, 1998; Bach et al., 2017; Diligenti et al., 2017), which allows symbolic reasoning rules to be satisfied in a graded rather than strictly binary manner, improving robustness when the input facts or intermediate predictions are uncertain.

Table 5: Ablation experiments on partial components of SSR with the Qwen2.5-7B-Instruct model. Where "- Filter" refers to the variant of SSR without the module for filtering obviously erroneous translated logical expressions, "- ECW" denotes the variant where the soft reasoning module of SSR assigns identical rather than entropy-based confidence weights to all conditions.

Dataset	- Filter	Standard	- ECW
PrOntoQA	97.0	**97.5**	**97.5**
Proofwriter	81.5	**86.0**	84.5
FOLIO	48.0	**64.5**	64.0
LogicalDeduction	68.5	**75.0**	**75.0**

Neuro-Symbolic Reasoning and LLM Auto-Formalization. Neuro-symbolic systems seek to combine the flexibility of neural language understanding with the rigor of symbolic reasoning (Hitzler et al., 2022; Hitzler and Sarker, 2022). A common pipeline involves translating natural-language premises into formal logic (*auto-formalization*), followed by symbolic inference using a solver. Representative approaches include directly solving translated logic programs (Pan et al., 2023), verifying LLM-generated reasoning chains step by step (Feng et al., 2025a), or perform knowledge graph reasoning via a differentiable model (Shengyuan et al., 2023). Despite their promise, these systems face a critical bottleneck in auto-formalization: The neural system can hardly avoid errors and incompleteness in the translation of natural language, whereas symbolic logic requires precise and complete formal representations. Minor errors in translation can easily render the symbolic problem unsatisfiable or misleading. Classical neuro-symbolic reasoning frameworks therefore either assume near-perfect translations or rely on repeated LLM-solver interaction to correct contradictions, which can be brittle or inefficient.

Although LLM-to-solver pipelines predate SSR, SSR uses solver outcomes to select among hard

inference, candidate-conditioned MaxSAT-style repair, and bounded ambiguity retrieval, and subsequently constructs replayable forward and backward traces. We therefore position SSR's contribution around the complementary and well-suited collaboration of these modules, rather than the general use of an LLM with a symbolic solver.

5 Conclusion

In this paper, we introduced Symbolic-Neural Soft Reasoning (SSR), a unified neuro-symbolic framework designed to address the fragility and readability limitations of neuro-symbolic systems. By integrating soft satisfiability constraints, SSR enables verifiable yet flexible reasoning under perturbations and fosters deeper interaction between symbolic and neural components. This design not only improves reasoning accuracy, but also supports the automatic generation of human-readable and logically verifiable reasoning chains for training and fine-tuning. Through extensive experiments, we demonstrate that SSR consistently outperforms existing neuro-symbolic and CoT-based approaches while maintaining a certain degree of robustness to added, removed, or modified perturbations during reasoning. Overall, we believe SSR provides a reliable and scalable foundation for advancing trustworthy reasoning and automatically obtain verifiable reasoning-chain data.

Acknowledgments

This research was supported by Beijing Natural Science Foundation (QY24035).

Limitations

Despite the efficacy of our SSR method, it still suffers from several limitations. First, LLMs are prone to generating erroneous formal logical representations during natural language formalization. While our designed soft protection mechanism mitigates such errors to a certain degree, it cannot eliminate them fundamentally. Future work will focus on further enhancing the reliability and precision of neural models in formal logic generation.

Second, SSR filters implausible formal logical statements and leverages dual-system interaction to compensate for missing conditions and commonsense knowledge. Nevertheless, its retrieval threshold is restricted to single conditional statements. Once exceeding this threshold, SSR defaults to standard CoT reasoning. Though this mechanism avoids symbolic reasoning collapse caused by excessive noise, it renders the model dependent on conventional reasoning paradigms and fails to fully utilize already acquired formal logical statements.

Third, while SSR can produce verifiable, automatically generated chain-of-thought data for LLM training, it inherits a typical drawback of neuro-symbolic systems: neural models may skip core reasoning steps rather than thoroughly internalizing symbolic reasoning logic.

Finally, the verifiability scope of SSR varies across reasoning execution branches. Hard-SMT answers are verifiable against the full translated program, relaxed answers are only verifiable against the maximum-weight retained subset, and retrieval-augmented answers rely additionally on LLM-validated explicit premises. None of these modes ensure semantic equivalence between LLM formalization and the original natural language problem. Furthermore, the canonical unary-predicate abstraction improves formalization stability at the cost of reduced expressiveness, targeted retrieval may introduce extrinsic errors, and CoT fallback cannot provide end-to-end symbolic reasoning guarantees.

Ethical Considerations

This paper aims to improve the reasoning performance and robustness of large language models by integrating neuro-symbolic modeling and soft reasoning, and to empower further development of deep neural networks through structured symbolic systems. From a societal perspective, our work is primarily applicable to areas such as trustworthy automated reasoning, AI planning, and AI for mathematics and science. We do not anticipate negative ethical or societal impacts arising from these applications, as the proposed methods are designed to enhance reliability and interpretability rather than to introduce new risks.

References

Janice Ahn, Rishu Verma, Renze Lou, Di Liu, Rui Zhang, and Wenpeng Yin. 2024. Large language models for mathematical reasoning: Progresses and challenges. *arXiv preprint arXiv:2402.00157*.

Mikhail L Arbuzov, Alexey A Shvets, and Sisong Beir. 2025. Beyond exponential decay: Rethinking error accumulation in large language models. *arXiv preprint arXiv:2505.24187*.

Stephen H Bach, Matthias Broecheler, Bert Huang, and Lise Getoor. 2017. Hinge-loss markov random fields

and probabilistic soft logic. *Journal of Machine Learning Research*, 18(109):1–67.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer.

Michelangelo Diligenti, Marco Gori, and Claudio Sacca. 2017. Semantic-based regularization for learning and inference. *Artificial Intelligence*, 244:143–165.

Didier Dubois and Henri Prade. 1998. Soft computing, fuzzy logic, and artificial intelligence. *Soft Computing*, 2(1):7–11.

Yu Feng, Nathaniel Weir, Kaj Bostrom, Sam Bayless, Darion Cassel, Sapana Chaudhary, Benjamin Kiesl-Reiter, and Huzefa Rangwala. 2025a. Vericot: Neuro-symbolic chain-of-thought validation via logical consistency checks. *arXiv preprint arXiv:2511.04662*.

Zhaopeng Feng, Yan Zhang, Hao Li, Bei Wu, Jiayu Liao, Wenqiang Liu, Jun Lang, Yang Feng, Jian Wu, and Zuozhu Liu. 2025b. Tear: Improving llm-based machine translation with systematic self-refinement. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 3922–3938.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured nlp tasks without finetuning. *arXiv preprint arXiv:2305.13971*.

Ahmad Ghazal, Tilmann Rabl, Minqing Hu, Francois Raab, Meikel Poess, Alain Crolotte, and Hans-Arno Jacobsen. 2013. Bigbench: Towards an industry standard benchmark for big data analytics. In *Proceedings of the 2013 ACM SIGMOD international conference on Management of data*, pages 1197–1208.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Simeng Han, Hailey Schoelkopf, Yilun Zhao, Zhenting Qi, Martin Riddell, Wenfei Zhou, James Coady, David Peng, Yujie Qiao, Luke Benson, and 1 others. 2024. Folio: Natural language reasoning with first-order logic. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22017–22031.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*.

Pascal Hitzler, Aaron Eberhart, Monireh Ebrahimi, Md Kamruzzaman Sarker, and Lu Zhou. 2022. Neuro-symbolic approaches in artificial intelligence. *National Science Review*, 9(6):nwac035.

Pascal Hitzler and Md Kamruzzaman Sarker. 2022. Neuro-symbolic artificial intelligence: The state of the art.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Angelika Kimmig, Stephen Bach, Matthias Broecheler, Bert Huang, Lise Getoor, Vikash Mansinghka, Daniel Roy, and Noah Goodman. 2012. A short introduction to probabilistic soft logic. In *Proceedings of the NIPS workshop on probabilistic programming: foundations and applications*, pages 1–4.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213.

Trung Quoc Luong, Xinbo Zhang, Zhanming Jie, Peng Sun, Xiaoran Jin, and Hang Li. 2024. Reft: Reasoning with reinforced fine-tuning. *arXiv preprint arXiv:2401.08967*.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. 2023. Logic-lm: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 3806–3824.

Kanghee Park, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D' Antoni. 2024. Grammar-aligned decoding. In *Advances in Neural Information Processing Systems*, volume 37, pages 24547–24568. Curran Associates, Inc.

Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki van Stein, and Thomas Back. 2024. Reasoning with large language models, a survey. *arXiv preprint arXiv:2407.11511*.

Abulhair Saparov and He He. 2022. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. *arXiv preprint arXiv:2210.01240*.

Zhenyi Shen, Hanqi Yan, Linhai Zhang, Zhanghao Hu, Yali Du, and Yulan He. 2025. Codi: Compressing chain-of-thought into continuous space via self-distillation. *arXiv preprint arXiv:2502.21074*.

Chen Shengyuan, Yunfeng Cai, Huang Fang, Xiao Huang, and Mingming Sun. 2023. Differentiable neuro-symbolic reasoning on large-scale knowledge graphs. *Advances in Neural Information Processing Systems*, 36:28139–28154.

Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, and 1 others. 2025. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.

Jun Sun, Yiteng Pan, and Xiaohu Yan. 2025a. Improving intermediate reasoning in zero-shot chain-of-thought for large language models with filter supervisor-self correction. *Neurocomputing*, 620:129219.

Zhongxiang Sun, Qipeng Wang, Haoyu Wang, Xiao Zhang, and Jun Xu. 2025b. Detection and mitigation of hallucination in large reasoning models: A mechanistic perspective. *arXiv preprint arXiv:2505.12886*.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. Proofwriter: Generating implications, proofs, and abductive statements over natural language. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3621–3634.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel Bowman. 2023. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. *Advances in Neural Information Processing Systems*, 36:74952–74965.

Yue Wan, Xiaowei Jia, and Xiang Lorraine Li. 2025. Unveiling confirmation bias in chain-of-thought reasoning. *arXiv preprint arXiv:2506.12301*.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Ke Weng, Lun Du, Sirui Li, Wangyue Lu, Haozhe Sun, Hengyu Liu, and Tiancheng Zhang. 2025. Autoformalization in the era of large language models: A survey. *arXiv preprint arXiv:2505.23486*.

An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, and 1 others. 2025. Qwen2. 5-1m technical report. *arXiv preprint arXiv:2501.15383*.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models, 2023. *URL https://arxiv. org/abs/2305.10601*, 3:1.

Bin Yu, Hang Yuan, Haotian Li, Xueyin Xu, Yuliang Wei, Bailing Wang, Weizhen Qi, and Kai Chen. 2025. Long-short chain-of-thought mixture supervised fine-tuning eliciting efficient reasoning in large language models. *arXiv preprint arXiv:2505.03469*.

Jiacheng Zhang, Huanbo Luan, Maosong Sun, Feifei Zhai, Jingfang Xu, and Yang Liu. 2021. Neural machine translation with explicit phrase alignment. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:1001–1010.

Zhanke Zhou, Rong Tao, Jianing Zhu, Yiwen Luo, Zengmao Wang, and Bo Han. 2024. Can language models perform robust reasoning in chain-of-thought prompting with noisy rationales? *Advances in Neural Information Processing Systems*, 37:123846–123910.

A Details of Logical Formalization

This appendix provides the technical details omitted from the main paper, including the formalization rules used for translating natural language into symbolic logic and the exact reasoning procedures employed by the SMT solver.

A.1 Object and Predicate Abstraction

Given a natural language statement, we instruct the LLM to identify all salient **objects**, defined as entities that possess attributes or participate in relations. Properties of objects are represented using a unary predicate abstraction $\text{Predicate}(\text{object})$.

Predicates are categorized according to their return types:

- **Numeric Sort** (`RealSort` and `IntSort`), which return numerical values and support arithmetic constraints (e.g., $\text{Age}(\text{Bob}) = 16$). The domain of these predicate variables is assumed to be a finite, enumerable subset of the real numbers, which in practice is usually instantiated as integers.

- **Boolean Sort** (`BoolSort`), which return truth values and represent states or conditions (e.g., $\text{Happy}(\text{Anne})$). Predicate variables of type `BoolSort` are restricted to the domain $\{\texttt{True}, \texttt{False}\}$. Boolean predicates follow open-world semantics. An explicitly asserted atom $P(c)$ is constrained to be true, while an explicit negation $\neg P(c)$ constrains it to be false. An atom absent from the translated facts remains unconstrained; its absence is not interpreted as either truth or falsity. This semantics allows SSR to return UNKNOWN when both the query and its negation are satisfiable.

All predicates are normalized to a consistent tense and grammatical number. For relations involving multiple objects, only the primary object appears as the predicate argument, while auxiliary objects are encoded in the predicate name. We adopted this single-object abstraction because preliminary use of multi-arity predicates produced frequent argument-order errors, such as $\text{Chase}(\textit{mouse}, \textit{cat})$ for "the cat chases the mouse", and non-entity arguments, such as $\text{Can}(\textit{bird}, \textit{fly})$. This choice improves formalization stability at the cost of longer predicate names and reduced expressiveness.

The parser can accept some syntactically valid multi-argument expressions as a compatibility fallback. However, the binary predicates shown in some Appendix E cases are verbatim LLM outputs that violate the canonical prompt format and are retained to illustrate formalization failures; they do not define a dataset-specific alternative representation.

A.2 Logic Operators and Precedence

The LLM is restricted to a predefined set of symbolic logical operators, including $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, $\forall$, and $\exists$.

Operator precedence follows standard first-order logic conventions, with parentheses used to override default evaluation order. The complete operator precedence table and syntax constraints are enforced during symbolic parsing to ensure compatibility with SMT solvers.

1. Numeric Predicates ($\text{Age}(\text{Bob})$)
2. Arithmetic Operators ($**, *, //, /, +, -$, arranged in accordance with operator precedence.)
3. Relational Operators ($<, >, \leq, \geq, =, \neq$)
4. Boolean Predicates ($\text{Happy}(\text{Anne})$)
5. Logical Negation ($\neg$)
6. Logical Conjunction ($\wedge$)
7. Logical Disjunction ($\vee$)
8. Implication ($\rightarrow$)
9. Quantifiers ($\forall, \exists$)

A.3 Output Structure

The LLM outputs a structured JSON object with three keys: `objects`, `facts`, and `query`, each mapping to a list of symbolic expressions represented as strings. This structured output is directly consumed by the symbolic solver.

B SMT-based Reasoning Procedures

We now formally describe how the symbolic facts and queries are processed by an SMT solver.

B.1 Satisfiability Modulo Theories

Let the set of logical facts be

$$\Phi = \varphi_1 \wedge \varphi_2 \wedge \cdots \wedge \varphi_n, \tag{5}$$

and let the query set be $Q = \{q_1, q_2, \ldots, q_m\}$.

Let the predicate variables appearing in Φ and Q be denoted by $[a_1, a_2, \ldots, a_k]$, where each a_j ranges over a domain $\mathcal{D}_j$. The Cartesian product

$$D = \mathcal{D}_1 \times \mathcal{D}_2 \times \cdots \times \mathcal{D}_k \tag{6}$$

represents the space of admissible assignments.

A finite set of formulas $\{p_1, \ldots, p_w\}$ is said to be *satisfiable* (**sat**) if there exists an assignment $\alpha \in D$ such that

$$p_1(\alpha) \wedge \cdots \wedge p_w(\alpha) = \texttt{True}. \tag{7}$$

Otherwise, it is *unsatisfiable* (**unsat**).

B.2 Boolean Query Entailment

For a closed Boolean query q_i, entailment is decided via two satisfiability checks:

$$\Phi \wedge q_i \quad \text{and} \quad \Phi \wedge \neg q_i.$$

The result is interpreted as follows:

- If $\Phi \wedge q_i$ is **sat** and $\Phi \wedge \neg q_i$ is **unsat**, then $\Phi \models q_i$.
- If $\Phi \wedge q_i$ is **unsat** and $\Phi \wedge \neg q_i$ is **sat**, then $\Phi \models \neg q_i$.
- If both are **sat**, the truth of q_i is undetermined under Φ.

Note that Φ itself must be satisfiable for any well-posed reasoning task. By the law of excluded middle, it is impossible for both $\Phi \wedge q_i$ and $\Phi \wedge \neg q_i$ to be unsatisfiable simultaneously.

B.3 Solving for Numeric Predicate Variables

If a query contains a numeric predicate variable a_i to be solved, we enumerate candidate values $a_i \in \{a_{i1}, a_{i2}, \ldots, a_{it_i}\}$. For each candidate value a_{ij}, we check whether $\Phi \wedge (a_i = a_{ij})$ is satisfiable.

The solution set is defined as

$$\mathrm{Sol}(a_i) = \{\, v \in \mathcal{D}_i \mid \Phi \wedge (a_i = v) \text{ is } \textbf{sat} \,\}. \tag{8}$$

B.4 Arithmetic Computation and Verification

When a problem consists solely of algebraic computations rather than formal logical reasoning, the framework effectively reduces to a specialized form of *Program-of-Thoughts*. In such cases, the LLM translates natural language descriptions into algebraic constraints, and the symbolic solver deterministically computes the solution. For example, consider the fact set

$$\text{Age(David)} = \text{Age(Peter)} + 3, \tag{9}$$

$$\text{Age(David)} - 10 = 2 \cdot (\text{Age(Peter)} - 10). \tag{10}$$

An SMT solver can compute the unique satisfying assignment. Additional commonsense constraints such as Age(David) > 0 and Age(Peter) > 0 can be incorporated to verify plausibility.

This satisfiability-based formulation also supports tasks such as automated theorem proving, counterexample construction, and feasible instance generation in combinatorial reasoning problems.

C SSR Soft Reasoning Algorithm

Below, we present the pseudocode for the SSR algorithm (Algorithm 1).

D Additional Experimental Details

D.1 Additional EXperimental Analyses

Formatting errors, including mismatched parentheses, commas, implication arrows, and natural-language fragments inside formal expressions, constitute the dominant failure type. FOLIO additionally produces many malformed predicates and missing conditions because of its longer and structurally more complex premises. LogicalDeduction is primarily affected by assignment errors, such as failing to recognize that "the second smallest among five objects" is equivalent to "the fourth largest". These results indicate that MaxSAT-style relaxation can repair some surface-level inconsistencies but cannot reliably recover omitted premises, incorrectly extracted objects, or mistranslated queries.

SSR is slower than Direct, CoT, and Logic-LM, although its aggregate runtime is slightly lower than VeriCoT. Table 7 shows that most of the cost comes from LLM formalization, while solver checking and relaxation are comparatively inexpensive. Thus, SSR should be viewed as improving the quality–verifiability trade-off rather than inference speed.

Table 6: Formalization-error attribution among 200 GPT-5.4 examples per dataset. Categories are non-exclusive, so their counts may exceed the number of failed examples. Predicate denotes malformed predicate construction, and Query denotes query mistranslation.

Dataset	Failed	Format	Predicate	Missing	Assignment	Query
PrOntoQA	0	0	0	0	0	0
ProofWriter	4	3	1	0	0	0
FOLIO	78	57	26	9	2	4
LogicalDeduction	22	5	0	0	18	0

Table 7: GPT-5.4 runtime breakdown in minutes for 200 examples per dataset. Components are rounded independently. A displayed value of 0.0 indicates that the cost is below the reported precision rather than that no checking or relaxation was performed.

Dataset	Total	Formalization	Check/Relax	Solve/Chain	Retrieval	CoT Fallback
PrOntoQA	31.4	25.9	0.0	5.4	0.0	0.0
ProofWriter	34.3	26.4	0.0	5.3	2.5	0.2
FOLIO	41.0	27.8	0.0	4.6	2.9	5.7
LogicalDeduction	33.5	26.0	0.0	5.1	0.8	1.6

D.2 Prompts Used in Experiments

This subsection reports the exact prompts used for different reasoning methods in our experiments to ensure reproducibility and fair comparison.

Direct Answer Prompt. For the direct reasoning baseline, we instruct the model to solve the problem and output only the final answer without intermediate reasoning:

```
You are a logical reasoning expert.
Solve the multiple-choice question and
↪ directly output the final answer
↪ format exactly as: '#### <Letter>'.
```

Chain-of-Thought (CoT) Prompt. For the CoT baseline, we encourage step-by-step reasoning while enforcing a strict final answer format:

```
You are a logical reasoning expert.
Solve the multiple-choice question step
↪ by step and explain your reasoning
↪ in a simple structured way.
Avoid overly long explanations. Provide
↪ clear reasoning followed by the
↪ final answer.
At the very end of your response, output
↪ the final answer format exactly as:
↪ '#### <Letter>'.
Example: '#### A' or '#### B'.
```

SSR Prompt for Boolean logical deduction Tasks. For SSR on Boolean logical deduction datasets, we prompt the model to translate the problem into a formal logical representation in JSON format:

```
You are a formal-logic translator.
Return ONLY a valid JSON object with
↪ exactly these keys: "objects",
↪ "facts", "query".
FORMALIZATION RULES:
1. "objects": All names of whose
↪ properties are described in the
↪ context. Note that group-referring
↪ nouns like "Zumpus" and "Vumpus" are
↪ predicates rather than objects. That
↪ is to say, words like "Dumpus" and
↪ "Rompus" cannot be used after
↪ predicates and cannot be added into
↪ "objects".
2. "facts": Each element must be a
↪ two-element list:
↪ ["<ORIGINAL_SENTENCE>",
↪ "<LOGICAL_FORM>"]. <LOGICAL_FORM>
↪ must be:
- An atomic predicate with exactly one
↪ object inside parentheses. All
↪ objects must be in the singular
↪ form.
- Use a prefix "not " for negation if
↪ and only if the natural language
↪ explicitly indicates negation. e.g.
↪ "not Happy(Anne)".
```

```
- Use "->" to represent the logical
↪  relationship of "if...then...", for
↪  example: ["If Anne is green then
↪  Anne is round.", "Green(Anne) ->
↪  Round(Anne)"]
- Use "and", "or" for a boolean
↪  combination of atomic predicates.
↪  e.g. "Have_a_cat(Bob) and
↪  Tall(Bob)", "not (Happy(Anne) or
↪  Sad(Anne))".
- For universal quantified laws where
↪  the object is not specified, use the
↪  form: "forall x. <premise> ->
↪  <conclusion>". Example: ["Tumpuses
↪  are wumpuses", "forall x. Tumpus(x)
↪  -> Wumpus(x)"], ["Every tumpus is
↪  large", "forall x. Tumpus(x) ->
↪  Large(x)"]
3. "query": a single string representing
↪  the formula to evaluate. Do NOT
↪  include the query in the "facts"
↪  list.
4. Permitted logical terms (only): and,
↪  or, ->, forall, exist, not, and
↪  Predicate(Object)
Translate the following context and
↪  question into logical language. For
↪  the "facts" section, the "original
↪  sentence" must be copied verbatim
↪  sentence by sentence in the exact
↪  order of the original context
↪  without any modifications; the
↪  "logical formalization" must be
↪  translated rigorously in accordance
↪  with the original sentences. Do not
↪  omit, add, or alter any conditions.
Context:
{context}
Question:
{question}
```

SSR Prompt for Numeric Logical Reasoning Tasks (LogicalDeduction). For numeric-based reasoning tasks like ordering and integer programming, we use a specialized formalization prompt with position-based predicates:

```
You are a formal logic translator.
Convert the following ordering puzzle
↪  into a JSON structure.
Return ONLY a valid JSON object with
↪  exactly these keys: "objects",
↪  "larger_direction", "facts",
↪  "query".
Requirements:
1. "objects": Copy the names of all the
↪  items to be compared directly as
↪  "objects". If the name of an object
↪  consists of multiple words, connect
↪  them with underscores.
2. "facts": Each element must be a
↪  two-element list:
↪  ["<ORIGINAL_SENTENCE>",
↪  "<LOGICAL_FORM>"]. <LOGICAL_FORM>
↪  must use only these predicates and
↪  syntax: >, <, =, and, or, not,
↪  Pos(object). Comparisons: Pos(A) <
↪  Pos(B), Pos(A) > Pos(B), Pos(A) = k.
↪  Any other symbols and predicates are
↪  prohibited.
3. "query": A dictionary in the form:
↪  {{"A": "<logical form of option A>",
↪  "B": "<logical form of option B>",
↪  "C": "<logical form of option C>",
↪  "D": "<logical form of option D>",
↪  "E": "<logical form of option E>"}}.
↪  Translate each option in the
↪  question into the corresponding
↪  logical language.
Note:
You must perform sorting comparisons in
↪  the following order: left < right,
↪  bottom < top, front < back and small
↪  < large (e.g., new < old, cheap <
↪  expensive).  For example, if there
↪  are 5 objects arranged from left to
↪  right and "apple is the second from
↪  the right", you should translate it
↪  as "Pos(apple) = 4" based on the
↪  total number of objects. All your
↪  comparison criteria must be
↪  consistent; additionally, pay
↪  attention to the impact of words
↪  like "less" and "more" on the
↪  translation into logical language.
Context
{context}
Question
{question}
Begin.
```

D.3 Perturbation Sentences Used in Robustness Experiments

To evaluate robustness under irrelevant or noisy inputs, we augment the original problem context with a fixed set of neutral perturbation sentences. These sentences are intentionally designed to be semantically self-contained, logically valid, and largely unrelated to the target reasoning task. They include general factual statements, commonsense knowledge, meta-level comments, numerical-information-based perturbations, and simple tautologies. Importantly, none of these perturbations introduces new task-relevant constraints or logical dependencies. This design ensures that any performance degradation arises from the sensitivity of a model to distracting information, rather than changes to the underlying logical structure of the problem. If not handled properly, these newly added sentences can introduce additional errors into SSR, as their corresponding logical translations may contain formatting errors, interfere with the LLM's identification of objects or other facts, or disrupt the retrieval process during automatic reasoning chain generation.

The 20 perturbation sentences used in our experiments are listed below:

- This sentence is included for completeness but does not affect the problem.
- The following text contains multiple independent statements.
- Logical reasoning tasks may vary in difficulty.
- Please note that not all sentences are equally important.
- Water boils at 100 degrees Celsius at sea level.
- The Earth completes one full rotation every 24 hours.
- A standard chessboard has 64 squares.
- Most modern buildings are made of concrete or steel.
- It was raining lightly outside the room.
- The room was quiet except for the sound of a ticking clock.
- After a long day, the reader might feel slightly tired.
- The experiment was conducted late in the afternoon.
- The meeting lasted for 45 minutes.
- There are seven continents on Earth.
- The building has three elevators and twelve floors.
- A triangle has exactly three sides.
- If today is Monday, then tomorrow is Tuesday.
- All squares are rectangles, but not all rectangles are squares.
- If A implies B, the reverse does not necessarily hold.
- Either the statement is true, or it is false.

E Case Studies and Further Discussions

Below, we provide several specific examples from the main experiment (Table 1) and a deeper analysis of the effectiveness of our SSR method.

E.1 PrOntoQA Dataset on Qwen-2.5-7B

Question: "Jompuses are large. Every jompus is a zumpus. Each zumpus is sweet. Zumpuses are numpuses. Every numpus is hot. Each tumpus is opaque. Numpuses are yumpuses. Most modern buildings are made of concrete or steel. Every yumpus is brown. Each yumpus is a wumpus. Wumpuses are not opaque. Wumpuses are impuses. Fae is a jompus. Is the following statement true or false? Fae is opaque."
Correct Answer: B (False)
LLM's Answer: B (False)
Objects: ["Fae"]
Facts: ["forall x. Jompus(x) → Large(x)", "forall x. Jompus(x) → Zumpus(x)", "forall x. Zumpus(x) → Sweet(x)", "forall x. Zumpus(x) → Numpus(x)", "forall x. Numpus(x) → Hot(x)", "forall x. Tumpus(x) → Opaque(x)", "forall x. Numpus(x) → Yumpus(x)", "forall x. (Modern(x) and Building(x)) → (Made_of_concrete(x) or Made_of_steel(x))", "forall x. Yumpus(x) → Brown(x)", "forall x. Yumpus(x) → Wumpus(x)", "forall x. Wumpus(x) → not Opaque(x)", "forall x. Wumpus(x) → Impus(x)", "Jompus(Fae)"]
Query: ["Opaque(Fae)"]
Sat_query: False
Sat_not_query: True
Forward: ["Jumpus(Fae) → Zumpus(Fae)", "Zumpus(Fae) → Numpus(Fae)", "Numpus(Fae) → Yumpus(Fae)", "Yumpus(Fae) → Wumpus(Fae)", "Wumpus(Fae) → not Opaque(Fae)"]
Correct_forward: True
Backward: ["not Opaque(Fae) ← Wumpus(Fae)", "Wumpus(Fae) ← Yumpus(Fae)", "Yumpus(Fae) ← Numpus(Fae)", "Numpus(Fae) ← Zumpus(Fae)", "Zumpus(Fae) ← Jumpus(Fae)"]
Correct_backward: True

Question: “Rompuses are spicy. Every rompus is an impus. Yumpuses are not small. Impuses are orange. Impuses are zumpuses. Zumpuses are not hot. Zumpuses are numpuses. Numpuses are metallic. Numpuses are wumpuses. Every wumpus is not kind. Each wumpus is a dumpus. Each dumpus is not bright. Every dumpus is a jompus. Jompuses are small. Jompuses are vumpuses. Each vumpus is not shy. Every vumpus is a tumpus. It was raining lightly outside the room. Alex is a zumpus. Is the following statement true or false? Alex is not small."

Correct Answer: B (False)

LLM's Answer: C (Unknown)

Objects: ["Alex", "Rompus", "Impus", "Yumpus", "Zumpus", "Numpus", "Wumpus", "Dumpus", "Jompus", "Vumpus", "Tumpus"]

Facts: ["Spicy(Rompus)", "forall x. Rompus(x) → Impus(x)", "not Small(Yumpus)", "Orange(Impus)", "forall x. Impus(x) → Zumpus(x)", "forall x. Zumpus(x) → not Hot(Zumpus)", "forall x. Zumpus(x) → Numpus(x)", "forall x. Numpus(x) → Metallic(Numpus)", "forall x. Numpus(x) → Wumpus(x)", "forall x. Wumpus(x) → not Kind(Wumpus)", "forall x. Wumpus(x) → Dumpus(x)", "forall x. Dumpus(x) → not Bright(Dumpus)", "forall x. Dumpus(x) → Jompus(x)", "forall x. Jompus(x) → Small(Jompus)", "forall x. Jompus(x) → Vumpus(x)", "forall x. Vumpus(x) → not Shy(Vumpus)", "forall x. Vumpus(x) → Tumpus(x)", "Raining_lightly_outside_the_room()", "Zumpus(Alex)"]

Query: ["not Small(Alex)"]

Sat_query: True

Sat_not_query: True

Forward: []

Correct_forward: False

Backward: []

Correct_backward: False

E.2 ProofWriter Dataset on Qwen-2.5-7B

Question: “The bear eats the tiger. The bear is not red. The bear likes the squirrel. Logical reasoning tasks may vary in difficulty. The bear visits the squirrel. The mouse eats the tiger. The mouse is red. The mouse is rough. The mouse likes the bear. The mouse does not like the tiger. The squirrel likes the bear. The tiger eats the mouse. If the mouse likes the bear and the bear likes the mouse then the mouse is not kind. If someone visits the mouse then they like the squirrel. If someone is green then they visit the bear. If someone likes the bear and the bear visits the tiger then they visit the bear. If someone eats the bear and they do not like the tiger then they are green. If someone visits the mouse then the mouse eats the bear. If someone is red and they eat the tiger then the tiger visits the mouse. If someone visits the bear then they are kind. Based on the above information, is the following statement true, false, or unknown? The tiger does not like the squirrel."

Correct Answer: B (False)

LLM's Answer: C (Unknown)

Objects: ["bear", "tiger", "squirrel", "mouse"]

Facts: ["Eats(bear, tiger)", "not Red(bear)", "Likes(bear, squirrel)", "Logical_reasoning_tasks_may_vary_in_difficulty.", "Visits(bear, squirrel)", "Eats(mouse, tiger)", "Red(mouse)", "Rough(mouse)", "Likes(mouse, bear)", "not Likes(mouse, tiger)", "Likes(squirrel, bear)", "Eats(tiger, mouse)", "forall x. (Likes(mouse, bear) and Likes(bear, mouse)) → not Kind(mouse)", "forall x. Visits(x, mouse) → Likes(x, squirrel)", "forall x. Green(x) → Visits(x, bear)", "forall x. (Likes(x, bear) and Visits(bear, tiger)) → Visits(x, bear)", "forall x. (Eats(x, bear) and not Likes(x, tiger)) → Green(x)", "forall x. Visits(x, mouse) → Eats(mouse, bear)", "forall x. (Red(x) and Eats(x, tiger)) → Visits(tiger, mouse)", "forall x. Visits(x, bear) → Kind(x)"]

Query: ["not Likes(tiger, squirrel)"]

(Continued on the next page)

LLM_CoT: “Let's break down the information and analyze the statement "The tiger does not like the squirrel" step by step. 1. **Direct Information:** - The bear eats the tiger. - The bear is not red. - The bear likes the squirrel. - The bear visits the squirrel. - The mouse eats the tiger. - The mouse is red. - The mouse is rough. - The mouse likes the bear. - The mouse does not like the tiger. - The squirrel likes the bear. - The tiger eats the mouse. 2. **Logical Deductions:** - From "The mouse likes the bear" and "The bear likes the squirrel," we can infer that the mouse likes the squirrel (since the bear likes the squirrel and the mouse likes the bear). - From "If the mouse likes the bear and the bear likes the squirrel then the mouse is not kind," we know that the mouse is not kind because it likes the bear and the bear likes the squirrel. - From "If someone visits the mouse then they like the squirrel," we know that the bear likes the squirrel because the bear visits the mouse. - From "If someone is green then they visit the bear," we know that being green is associated with visiting the bear. - From "If someone likes the bear and the bear visits the tiger then they visit the bear," we know that liking the bear and the bear visiting the tiger implies visiting the bear. - From "If someone eats the bear and they do not like the tiger then they are green," we know that eating the bear and not liking the tiger implies being green. - From "If someone visits the mouse then the mouse eats the bear," we know that visiting the mouse implies the mouse eats the bear. - From "If someone is red and they eat the tiger then the tiger visits the mouse," we know that being red and eating the tiger implies the tiger visits the mouse. - From "If someone visits the bear then they are kind," we know that visiting the bear implies being kind. 3. **Analyzing the Statement:** - We need to determine if the tiger does not like the squirrel. - We know the bear likes the squirrel. - The bear visits the squirrel. - The mouse likes the bear. - The mouse does not like the tiger. - The squirrel likes the bear. - The tiger eats the mouse. 4. **Connecting the Dots:** - The bear likes the squirrel. - The mouse likes the bear. - The tiger eats the mouse. - There is no direct information about the tiger's feelings towards the squirrel. 5. **Conclusion:** - We do not have any direct or indirect information that tells us whether the tiger likes or dislikes the squirrel. - Therefore, based on the given information, we cannot determine if the tiger does not like the squirrel. #### C”

Forward: []

Correct_forward: False

Backward: []

Correct_backward: False

Question: “Bob is rough. Charlie is nice. Charlie is rough. Charlie is white. Dave is not white. Gary is cold. Gary is quiet. The following text contains multiple independent statements. Rough, white things are quiet. If Charlie is round and Charlie is quiet then Charlie is rough. All young things are round. Nice things are white. Rough,

nice things are cold. If something is round and rough then it is nice. If something is rough then it is young. Based on the above information, is the following statement true, false, or unknown? Gary is not white."

Correct Answer: C (Unknown)

LLM's Answer: C (Unknown)

Objects: ["Bob", "Charlie", "Dave", "Gary"]

Facts: ["Rough(Bob)", "Nice(Charlie)", "Rough(Charlie)", "White(Charlie)", "not White(Dave)", "Cold(Gary)", "Quiet(Gary)", "forall x. (Rough(x) and White(x)) → Quiet(x)", "forall x. (Round(x) and Quiet(x)) → Rough(x)", "forall x. Young(x) → Round(x)", "forall x. Nice(x) → White(x)", "forall x. (Rough(x) and Nice(x)) → Cold(x)", "forall x. (Round(x) and Rough(x)) → Nice(x)", "forall x. Rough(x) → Young(x)"]

Query: ["not White(Gary)"]

Sat_query: True

Sat_not_query: True

Forward: (Terminate before reaching the query according to the procedure)

Correct_forward: True

Backward: ["White(Gary) ← Nice(Gary)", "Nice(Gary) ← (Round(Gary) and Rough(Gary))", "Round(Gary) ← Young(Gary)", "Young(Gary) ← Rough(Gary)", "Rough(Gary) ← (Round(Gary) and Quiet(Gary))", "Round(Gary) ← Young(Gary)", ...] (infinite loop)

Correct_backward: False

E.3 FOLIO Dataset on Qwen-2.5-7B

Question: ""There are six types of wild turkeys: Eastern wild turkey, Osceola wild turkey, Gould's wild turkey, Merriam's wild turkey, Rio Grande wild turkey, and Ocellated wild turkey. Most modern buildings are made of concrete or steel. Tom is not an Eastern wild turkey. Tom is not an Osceola wild turkey. Tom is also not a Gould's wild turkey, or a Merriam's wild turkey, or a Rio Grande wild turkey. Tom is a wild turkey. Based on the above information, is the following statement true, false, or uncertain? Tom is an Ocellated wild turkey.""

Correct Answer: A (True)

LLM's Answer: A (True)

Objects: ["Tom"]

Facts: ["forall x. Wild_turkey(x) → (Eastern_wild_turkey(x) or Osceola_wild_turkey(x) or Goulds_wild_turkey(x) or Merriams_wild_turkey(x) or Rio_Grande_wild_turkey(x) or Ocellated_wild_turkey(x))", "forall x. (Modern_buildings(x) → (Concrete(x) or Steel(x)))", "not Eastern_wild_turkey(Tom)", "not Osceola_wild_turkey(Tom)", "not Goulds_wild_turkey(Tom) and not Merriams_wild_turkey(Tom) and not Rio_Grande_wild_turkey(Tom)", "Wild_turkey(Tom)"]

Query: ["Wild_turkey(Tom) and Ocellated_wild_turkey(Tom)"]

Sat_query: True

Sat_not_query: False

Forward: ["Wild_turkey(Tom) → (Eastern_wild_turkey(Tom) or Osceola_wild_turkey(Tom) or Goulds_wild_turkey(Tom) or Merriams_wild_turkey(Tom) or Rio_Grande_wild_turkey(Tom) or Ocellated_wild_turkey(Tom))", ""not Eastern_wild_turkey(Tom)", "not Osceola_wild_turkey(Tom)", "not Goulds_wild_turkey(Tom) and not Merriams_wild_turkey(Tom) and not Rio_Grande_wild_turkey(Tom)" → Ocellated_wild_turkey(Tom)"]

Correct_forward: True

Backward: ["Wild_turkey(Tom) ← Wild_turkey(Tom)", "Ocellated_wild_turkey(Tom) ← "forall x. Wild_turkey(x) → (Eastern_wild_turkey(x) or Osceola_wild_turkey(x) or Goulds_wild_turkey(x) or Merriams_wild_turkey(x) or Rio_Grande_wild_turkey(x) or Ocellated_wild_turkey(x))""] (Terminated)

Correct_backward: False

Question: ""A person is either a Grand Slam champion or an Oscar-nominated actor. All people who are Grand Slam champions are professional tennis players. All Oscar-nominated actors are celebrities. All professional tennis players are athletes. If a person is a celebrity then they are well paid. If a person is an athlete then they are famous. All well-paid people live in tax havens. If Djokovic is famous and is an athlete, then Djokovic lives in well paid. If today is Monday, then tomorrow is Tuesday. Based on the above information, is the following statement true, false, or uncertain? Djokovic lives in a tax haven.""

Correct Answer: A (True)

LLM's Answer: C (Unknown)

Objects: ["Djokovic"]

Facts: ["forall x. (Grand_Slam_champion(x) or Oscaro_nominated_actor(x))", "forall x. Grand_Slam_champion(x) → Professional_tennis_player(x)", "forall x. Oscaro_nominated_actor(x) → Celebrity(x)", "forall x. Professional_tennis_player(x) → Athlete(x)", "forall x. Celebrity(x) → Well_paid(x)", "forall x. Athlete(x) → Famous(x)", "forall x. Well_paid(x) → Lives_in_tax_havens(x)", "forall x. (Famous(x) and Athlete(x)) → Lives_in_well_paid(x)", "forall x. Today_is_Monday(x) → Tomorrow_is_Tuesday(x)"]

Query: ["Lives_in_tax_havens(Djokovic)"]

Sat_query: True

Sat_not_query: True

Discriminator: ["Oscaro_nominated_actor_Djokovic"]

Retrieval_discriminator: None

Forward: (Terminate before reaching the query according to the procedure)

Correct_forward: False

Backward: ["Lives_in_tax_havens(Djokovic) ← Well_paid(Djokovic)", "Well_paid(Djokovic) ← Celebrity(Djokovic)", "Celebrity(Djokovic) ← Oscaro_nominated_actor(Djokovic)", "Oscaro_nominated_actor(Djokovic) ← "forall x. (Grand_Slam_champion(x) or Oscaro_nominated_actor(x))""] (Terminated)

Correct_backward: False

E.4 LogicalDeduction Dataset on Qwen-2.5-7B

Question: "Either the statement is true, or it is false. The following paragraphs each describe a set of five objects arranged in a fixed order. The statements are logically consistent within each paragraph. Please note that not all sentences are equally important. On a branch, there are five birds: a cardinal, a robin, a blue jay, a quail, and a raven. The robin is to the right of the raven. The cardinal is the leftmost. It was raining lightly outside the room.

The raven is to the right of the blue jay. If A implies B, the reverse does not necessarily hold. The blue jay is the third from the left. A) The cardinal is the second from the right. B) The robin is the second from the right. C) The blue jay is the second from the right. D) The quail is the second from the right. E) The raven is the second from the right."

Correct Answer: E (The raven is the second from the right)

LLM's Answer: E (The raven is the second from the right)

Objects: ["cardinal", "robin", "blue_jay", "quail", "raven"]

Facts: ["Pos(robin) > Pos(raven)", "Pos(cardinal) = 1", "Pos(raven) > Pos(blue_jay)", "Pos(blue_jay) = 3"]

Query: ["Pos(cardinal) = 4", "Pos(robin) = 4", "Pos(blue_jay) = 4", "Pos(quail) = 4", "Pos(raven) = 4"]

Forward: [""Pos(blue_jay) = 3", "Pos(robin) > Pos(raven)", "Pos(raven) > Pos(blue_jay)", → Pos(raven) = 4"]

Correct_forward: True

Backward: ["Pos(raven) = 4 ← "Pos(robin) > Pos(raven)", "Pos(raven) > Pos(blue_jay)"", "Pos(robin) > Pos(raven) ← "Pos(robin) = 5"", "Pos(robin) = 5 ← sat", "Pos(raven) > Pos(blue_jay) ← "Pos(blue_jay) = 3""]

Correct_backward: True

Question: "If today is Monday, then tomorrow is Tuesday. The following paragraphs each describe a set of five objects arranged in a fixed order. Water boils at 100 degrees Celsius at sea level. The statements are logically consistent within each paragraph. Either the statement is true, or it is false. On a branch, there are five birds: a cardinal, a hawk, a hummingbird, a raven, and an owl. The raven is to the left of the hummingbird. The meeting lasted for 45 minutes. The hawk is to the left of the owl. The cardinal is to the left of the hawk. The raven is the second from the right. A) The cardinal is the second from the left. B) The hawk is the second from the left. C) The hummingbird is the second from the left. D) The raven is the second from the left. E) The owl is the second from the left."

Correct Answer: B (The hawk is the second from the left)

LLM's Answer: A (The cardinal is the second from the left)

Objects: ["cardinal", "hawk", "hummingbird", "raven", "owl"]

Facts: ["Pos(raven) < Pos(hummingbird)", "Pos(hawk) < Pos(owl)", "Pos(cardinal) < Pos(hawk)", "Pos(raven) = 4"]

Query: ["Pos(cardinal) = 1", "Pos(hawk) = 1", "Pos(hummingbird) = 1", "Pos(raven) = 1", "Pos(owl) = 1"]

Forward: [""Pos(raven) < Pos(hummingbird)", "Pos(raven) = 4" → Pos(hummingbird) = 5", ""Pos(hawk) < Pos(owl)", "Pos(cardinal) < Pos(hawk)", "Pos(raven) = 4" → Pos(cardinal) = 1"]

Correct_forward: False

Backward: []

Correct_backward: False

E.5 Failure Analysis of SSR on Enhancing Reasoning Accuracy

For conciseness and to showcase a diverse set of successful and failed reasoning cases, the examples presented in this section are not randomly selected. We observe that most failures of SSR are closely associated with errors in the LLM-generated formal logical translations. These failures primarily arise from the following causes: (1) incorrect extraction of objects, which subsequently leads to biased interpretation of universally quantified formulas (e.g., the second PrOntoQA example); (2) malformed translations of predicate functions, resulting in invalid or misinterpreted logical expressions (e.g., the first ProofWriter example); (3) improper handling of injected perturbation sentences, which interferes with fact parsing or retrieval (e.g., the first ProofWriter example); (4) lack of alignment between logically equivalent predicate forms expressed with different surface representations (e.g., the second FOLIO example); and (5) incorrect translation of the query itself, causing the downstream reasoning target to be entirely mis-specified (e.g., the second LogicalDeduction example). Although soft-logic relaxation in SSR can mitigate a subset of minor perturbations and translation inconsistencies, the diversity of problem formulations and the inherent stochasticity of LLM outputs make it difficult to fully eliminate all such failure cases.

From a broader perspective, these failure modes can be roughly divided into two categories. Errors caused by surface-level inconsistencies in logical translation, such as malformed predicates, misaligned but semantically equivalent predicate names, or mild interference from perturbation sentences, are in principle repairable through stronger normalization (Zhang et al., 2021), constrained decoding (Park et al., 2024; Geng et al., 2023), or post-hoc correction mechanisms (Feng et al., 2025b). In contrast, failures stemming from fundamentally incorrect object extraction or mis-specified queries reflect deeper semantic misunderstandings of the problem statement and remain a more systemic challenge. Addressing these issues likely requires improvements in the reliability of LLM-based semantic parsing or tighter coupling between translation and symbolic verification, which we leave for future work.

E.6 Failure Analysis of Automatic Reasoning Chain Generation

We further analyze failure cases where SSR fails to automatically generate a coherent reasoning chain, even when it is able to reach a correct or partially correct conclusion. These failures mainly arise from limitations in the retrieval-based forward and

backward reasoning procedures. First, incorrect extraction of objects, query or malformed facts can prevent both forward and backward reasoning from performing valid retrieval, as the necessary predicates or relations cannot be matched correctly (e.g., the second PrOntoQA example, the first ProofWriter example, and the second LogicalDeduction example). Second, during backward reasoning, the retrieval process may fail to leverage additional information for elimination, causing the algorithm to enter a dead loop when repeatedly querying the same pair of predicate functions (e.g., the second ProofWriter example). Third, backward reasoning may encounter bottlenecks in problems that require extensive elimination over a large space of candidate values, leading to incomplete or excessively long reasoning chains (e.g., the first FOLIO example). Finally, when the LLM-generated logical translation omits critical conditions, the retrieval-based reasoning process lacks sufficient premises to proceed, resulting in premature termination of the reasoning chain (e.g., the second FOLIO example). These cases highlight that successful chain construction not only depends on correct symbolic inference, but also critically relies on the completeness and structural quality of the underlying logical representation.

Algorithm 1 Symbolic-Neural Soft Reasoning (SSR)

Input: Fact set $\Phi = \{\varphi_1, \ldots, \varphi_n\}$ translated from LLM, query q
Output: Answer to query q
// Syntax-level filtering
For each $\varphi \in \Phi$:
    If φ is syntactically malformed, replace it with $\texttt{BoolVal}(\texttt{True})$.
If the number of $\texttt{BoolVal}(\texttt{True}) > 1$, or $q = \texttt{BoolVal}(\texttt{True})$:
    return fallback to CoT-based reasoning.
// Confidence-weighted softening
For each $\varphi \in \Phi$, assign a confidence weight $w(\varphi) > 0$.
// Enumerate candidate query values
Let $\mathcal{Q} = \{q_1, q_2, \ldots\}$ be the candidate values for q.
For each $q_j \in \mathcal{Q}$:
    Check satisfiability of $\Phi \wedge (q = q_j)$
    Record the result as sat or unsat.
// Case I: exactly one satisfiable result
If exactly one q_j yields sat:
    return $q = q_j$.
// Case II: no satisfiable result
If no q_j yields sat:
    For each $q_j \in \mathcal{Q}$, find a subset $\Phi'_j \subseteq \Phi$ maximizing
    $\sum_{\varphi \in \Phi'_j} w(\varphi)$ such that $\Phi'_j \wedge (q = q_j)$ is sat.
    Select the q_j with the maximum total weight.
    If multiple candidates tie, **return** unknown;
    otherwise, **return** $q = q_j$.
// Case III: multiple satisfiable results
While multiple q_j yield sat:
    Enumerate candidate predicate values.
    For each candidate value:
        If fixing $a =$ value yields exactly one sat result,
        replace it by a stronger derivable assignment if possible and continue the iteration.
    Stop when no further refinement is possible.
Ask the LLM to verify whether the derived predicate assignments $a =$ value follow from the problem statement or commonsense.
If the LLM confirms:
    return the unique solution;
Otherwise:
    return unknown.